Question 1 : Odd Bounty

Logic:
This problem reduces to identifying the *unique element* among three values where exactly two are equal and one is different.

Since two pirates share the same bounty, that value appears **twice**, and the rival pirate's bounty appears **once**.
So, the task is to detect the value that is **not equal to the other two**.

There are only three possibilities:

- If a == b, then both belong to the same crew → the rival is c.
- If a == c, then both belong to the same crew → the rival is b.
- Otherwise (b == c), the rival is a.

Approach:

For each test case:

1. Read the three integers a, b, c.
2. Compare values:
    - If a == b → output c
    - Else if a == c → output b
    - Else → output a
3. Repeat for all t test cases.

Alternative Insight (XOR Trick):
You can also use bitwise XOR:

- a ^ b ^ c cancels out the duplicate values and leaves the unique one.

Example:

- 0 ^ 1 ^ 0 = 1
- 8 ^ 3 ^ 3 = 8

Solution in C++:

```
#include <bits/stdc++.h>
```

```
using namespace std;

int main() {
    int t;
    cin>>t;
    while(t--) {
            int a,b,c;
        cin>>a>>b>>c;
        cout<<a^b^c<<endl;
    }
    return 0;
}
```

Time Complexity: O(t) (constant work per test case).

Space Complexity: O(1).

Question 2 : Sanji's Spinning Menu Disaster

Logic:

This problem is about finding the **minimum number of rotations** required to type a string using a circular wheel of lowercase English letters.

The wheel contains letters from 'a' to 'z' arranged in a circle.
Sanji starts at 'a' and must reach each character of the given string.

For each move:

- You can rotate **clockwise** or **counterclockwise**
- Choose the **minimum distance**

Since the wheel is circular:

- Direct distance = |current - previous|
- Reverse distance = 26 - direct distance
- Minimum rotation = min(direct, reverse)

Approach:

For the given string:

Initialize:

- ans = 0 (total rotations)
- prev = 'a' (starting position)

Traverse the string:

- For each character c:
    - Compute diff = abs(c - prev)
    - Compute step = min(diff, 26 - diff)
    - Add step to ans
    - Update prev = c

Output ans.

Solution in C++:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    string s;
    cin >> s;

    int ans = 0;
    char prev = 'a';

    for(char c : s) {
        int diff = abs(c - prev);
        int step = min(diff, 26 - diff);
        ans += step;
```

```
        prev = c;
    }

    cout << ans << endl;
    return 0;
}
```

Time Complexity: O(n)

The string is traversed once, and each character requires constant time computation.

Space Complexity: O(1)

Only a few variables are used. No extra space proportional to input size is required.

Question 3 : Vegapunk's Keyboard Chaos

Logic:

This problem is about **mapping characters between two keyboard layouts**.

We are given:

- First layout → original keyboard used
- Second layout → intended keyboard
- A string typed using the first layout

Each character typed corresponds to a **position in the first layout**, and we must replace it with the character at the **same position in the second layout**.

Key observations:

- Lowercase letters → directly map
- Uppercase letters → map their lowercase equivalent, then convert back to uppercase
- Digits → remain unchanged

Approach:

1. Read:
    - s1 → first layout
    - s2 → second layout
    - s → typed string
2. Create a mapping:
    - For each index i from 0 to 25:
        - Map s1[i] → s2[i]
3. Traverse string s:
    - If character is **lowercase**:
        - Replace using mapping
    - If character is **uppercase**:
        - Convert to lowercase
        - Map it
        - Convert result back to uppercase
    - If character is **digit**:
        - Keep it unchanged
4. Print the final transformed string

Solution in C++:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
   string s1, s2, s;
   cin >> s1 >> s2 >> s;

   map<char, char> mp;

   // Create mapping
   for(int i = 0; i < 26; i++) {
      mp[s1[i]] = s2[i];
   }
```

```
    string result = "";

    for(char c : s) {
        if(islower(c)) {
            result += mp[c];
        }
        else if(isupper(c)) {
            char lower = tolower(c);
            char mapped = mp[lower];
            result += toupper(mapped);
        }
        else {
            result += c; // digits remain unchanged
        }
    }

    cout << result << endl;
    return 0;
}
```

Time Complexity: O(n)

- Mapping creation takes constant time (26 operations)
- Traversing the string takes O(n)

Space Complexity: O(1)

- Mapping stores only 26 characters
- Constant extra space used

Question 4 : Luffy's Food Strategy

Logic :

Luffy must feed his crew for N days. Each day has an individual food cost Fi.

Luffy also has an alternative option:

- **Buy a Marine Ration Pack**

- Cost = P berries
- Covers D consecutive days

- Cost remains P, no matter how expensive those days are.

Key Observation:

For any group of D days:

- Paying individually costs = sum of food costs
- Using ration pack costs = P

So, for every group, we should choose:

minimum(sum of D days, P)

Greedy Strategy:
To minimize total expense:

1. Sort food costs in descending order

- Expensive days should be grouped together.
- This maximizes the benefit of using ration packs.

2. Group days into blocks of size D

- First D most expensive days → Block 1
- Next D days → Block 2

  and so on…

3. For each block:

- Compute total individual cost.
- Compare with ration pack price P.
- Add the cheaper option to answer.

Handle the last group (if size < D) similarly.

This greedy works because ration packs give maximum savings when applied to the costliest days first.

Solution in C++ :

```
#include <bits/stdc++.h>

using namespace std;


#define fastio ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
```

```
#define ll long long
#define all(x) (x).begin(), (x).end()
#define pb push_back

int main() {
    fastio;

    ll n, d, p;
    cin >> n >> d >> p;

    vector<ll> vec(n);
    for(auto &i : vec) cin >> i;

    // Sort in descending order
    sort(vec.rbegin(), vec.rend());

    vector<ll> temp;
    ll sum = 0LL, ans = 0LL;

    // Create groups of size D
    for(int i = 1; i <= n; i++){
        sum += vec[i - 1];

        if(i % d == 0){
            temp.push_back(sum);
            sum = 0LL;
        }
    }
```

```
    // Remaining days
    if(n % d != 0)
        temp.push_back(sum);

    // Choose minimum cost for each group
    for(auto i : temp)
        ans += min(i, p);

    cout << ans << endl;
    return 0;
}
```

Time Complexity : O (n log n)

Sorting dominates the complexity.

Space Complexity : O(n)

Storage of food cost array and grouping vector.

Question 5 : Luffy and the Broken Meat Sticks

Logic :

Originally, every meat stick had the same length L.

After Luffy's chaos, each stick became either:

- Unbroken → one piece of length L, or
- Broken into exactly two pieces whose lengths:
    - are positive integers
    - sum to L

Finally, we are given N pieces with lengths:

$$A1,A2,.....,An$$

We must find all possible values of original length L.

Approach:

After sorting the pieces:

- Any piece equal to L represents an intact stick.
- All remaining smaller pieces must be paired such that:

$$A_i + A_j = L$$

because broken sticks produce exactly two parts.

<u>Possible Candidates for L:</u>

Only two values can ever work:

1. Largest piece itself

$$L = \text{max element}$$

Means largest sticks remained intact.

2. Sum of smallest and largest piece

$$L = \text{min element} + \text{max element}$$

Smallest and largest may come from a broken stick.

No other value of L is possible.

<u>Validation Strategy:</u>

For a chosen target L:

1. Remove all elements equal to L (intact sticks).

2. Remaining pieces must:

- be even in count
- pair symmetrically:

$$vec[i] + vec[n-1-i] = L$$

3. If all pairs satisfy this → L is valid.

Solution in C++ :

```
#include <bits/stdc++.h>

using namespace std;
```

```
#define fastio ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
#define ll long long
#define all(x) (x).begin(), (x).end()
#define pb push_back

bool check(vector<ll>& vec , ll target){
    ll n = vec.size();

    // Remove intact sticks
    while(n > 0 && vec[n - 1] == target)
        n--;

    // Remaining pieces must be paired
    if(n % 2 != 0)
        return false;

    for(int i = 0; i < n; i++){
        if(vec[i] + vec[n - 1 - i] != target)
            return false;
    }
    return true;
}

int main() {
    fastio;

    ll n;
    cin >> n;
```

```
    vector<ll> vec(n);
    for(auto &i : vec)
        cin >> i;

    sort(all(vec));

    // Candidate 1: Largest element
    if(check(vec, vec[n - 1]))
        cout << vec[n - 1] << " ";

    // Candidate 2: Smallest + Largest
    if(check(vec, vec[0] + vec[n - 1]))
        cout << vec[0] + vec[n - 1] << "\n";

    return 0;
}
```

Time Complexity : O(n log n)

Sorting dominates.

Each validation check runs in linear time.

Space Complexity : O(n)

Storage of the meat pieces array.